arXiv:submit/6168033 [cs.SE] 31 Jan 2025

REPOAUDIT: An Autonomous LLM-Agent for Repository-Level Code Auditing

Jinyao Guo* [1] **Chengpeng Wang*** [1] **Xiangzhe Xu** [1] **Zian Su** [1] **Xiangyu Zhang** [1]

Abstract

Code auditing is a code review process with the goal of finding bugs. Large Language Models (LLMs) have shown substantial potential in this task, offering the ability to analyze programs without compilation and enabling customized bug detection following specified prompts. However, applying LLMs to repository-level code auditing presents notable challenges. The inherent context limits and hallucinations of LLMs can lead to the low quality of bug reports. Meanwhile, the large size of software repositories introduces substantial time and token costs, hindering efficiency and scalability in real-world scenarios.

This work introduces an autonomous LLM-agent, REPOAUDIT, designed to enable precise and efficient repository-level code auditing. Equipped with the agent memory, REPOAUDIT explores the code repository on demand, analyzing data-flow facts along different feasible program paths in individual functions. It also introduces the validator to check the data-flow facts for hallucination mitigation and examine the satisfiability of path conditions of potential buggy paths, which enables REPOAUDIT to discard false positives in the code auditing. Our experiment shows that REPOAUDIT powered by Claude 3.5 Sonnet successfully finds 38 true bugs in 15 real-world systems, consuming 0.44 hours and $2.54 per project on average.

1. Introduction

The rapid innovation of large language models (LLMs) has remarkably enhanced the productivity of software developers (Wang et al., 2021; Rozière et al., 2023; Guo et al., 2024). LLM-powered IDE plugins, such as Copilot, enable agile code generation by facilitating flexible interactions with LLMs (Barke et al., 2023). However, in the era of LLMs, auditing the rapidly expanding codebase presents a more formidable challenge than writing code. Traditional program analysis techniques, such as dynamic and static analysis, focus primarily on observing runtime behaviors or symbolically reasoning about intermediate code generated during compilation (King, 1976; Cadar et al., 2008; Calcagno et al., 2009; Sui and Xue, 2016; Shi et al., 2018a). Unfortunately, software systems are often non-executable and even uncompilable during the human-LLM collaborative development phase. Moreover, existing analysis techniques demand substantial expertise, such as a deep understanding of compiler internals (Zhang et al., 2024a; Zhou et al., 2024). Therefore, current program analysis approaches often fall short in meeting the practical requirements of code auditing in real-world contexts (Johnson et al., 2013).

In recent years, extensive research has focused on leveraging LLMs for code auditing via prompt engineering (Fang et al., 2024a; Hao et al., 2023; Ding et al., 2024). Unlike traditional analysis techniques, LLM-driven code auditing directly analyzes source code without program compilation or execution. By describing analysis requirements through natural language and few-shot examples in prompts, the customization of analysis is also significantly simplified. However, many existing LLM-driven code auditing techniques are largely restricted to small-scale codebases, such as smart contracts (Sun et al., 2024; Zhang and Zhang, 2024), and lack the capability to support repository-level code auditing in complex, real-world scenarios.

Direct Prompting Hardly Works. A straightforward solution is to break down a repository into smaller pieces and prompt the model with individual pieces. This approach often falls short for non-local bugs, which may require reasoning across a large number of interconnected code snippets spanning multiple functions, classes, and files. Even with significant future advancements in model reasoning capacity, the fundamental differences between programs and natural language texts for which transformer models were designed make LLMs insufficient for comprehensive repository analysis. In particular, a code repository can be conceptualized as an enormous graph, with nodes representing individual statements and edges capturing intricate relationships such as control-flow, data-flow, and other interdependencies between statements. These relationships are precisely defined and immensely complex at the reposi-

*Equal contribution [1]Purdue University, West Lafayette, IN, 47906, USA. Correspondence to: Jinyao Guo <guo846@purdue.edu>, Chengpeng Wang <wang6590@purdue.edu>.

tory level. For instance, the number of data-flow edges in the project `shadowsocks-libev` exceeds one million, significantly surpassing the complexity of implicit graphical structure present in any LLM's pre-training data sample. Such distribution differences render direct prompting ineffective as shown by our experiments in Section 2.2.

In addition, detecting many types of bugs requires reasoning about properties along specific program paths. For instance, a memory leak occurs when allocated memory is not freed along some program path. Detecting such so-called *path-sensitive* bugs (Shi et al., 2018a) necessitates unfolding the program's graph structure into individual paths and analyzing these paths one by one. However, this leads to the well-known *path explosion* problem, as the number of paths grows exponentially with the number of statements. Hence, direct prompting is akin to presenting a large project on an enormous screen and expecting a human auditor to identify bugs along complex and lengthy paths solely by reading and interpreting the code—a task highly unlikely to succeed.

Human Auditing. In practice, human auditors do not operate in such a manner. As revealed by existing cognitive science literature (Anicic et al., 2012), humans are highly effective at reasoning about events that occur in order, e.g., in temporal or spatial order. Human auditors hence tend to explore complex graphical structures in code by following paths that denote the execution order. To avoid excessive exploration, they traverse only a subset of paths most relevant to the targeted property, leveraging implicit abstraction to preclude irrelevant paths. An example of detecting null pointer dereference can be found in Section 2.1.

Our Solution. It is widely believed that LLMs operate similarly to humans but with significantly greater "endurance" and access to a broader spectrum of knowledge (Li et al., 2023; Long et al., 2024; Qian et al., 2024). Building on this perspective, we propose a novel LLM-based repository auditing agent, named REPOAUDIT, inspired by human auditing practices. REPOAUDIT addresses the fundamental misalignment between LLMs' tendency to reason sequentially and the inherently complex graphical structures of software repositories through path-sensitive and demand-driven graph traversal. By leveraging LLMs' abstraction capabilities, REPOAUDIT mitigates path explosion by excluding irrelevant code regions and sub-paths. Also, it minimizes inherent hallucinations by sanitizing final outputs through the validation of several well-formedness properties.

More specifically, the agent REPOAUDIT consists of three components, including the initiator, the explorer, and the validator. First, the initiator identifies starting points based on the properties under investigation, such as `NULL` values when scanning for null pointer dereference bugs. Second, the explorer traverses the relevant functions on demand. Similar to how human auditors analyze code function by function, the explorer starts by querying the LLMs with the functions containing these starting points. Instead of explicitly and programmatically enumerating individual paths inside a function, as in traditional compiler-based automated scanners, the explorer leverages LLMs' inherent ability to implicitly distinguish relevant paths from the irrelevant ones and only reasons about the former. If invocations to other functions and returns within a function are considered relevant after analyzing paths within the function—for instance, when null pointer values propagate through these function boundaries—the system synthesizes follow-up prompts to extend the scanning into callee or caller functions as needed. Third, REPOAUDIT checks the output of the explorer before storing it in the agent memory and also examines the bug report candidates by examining the path conditions of the buggy program paths. This validation design can significantly improve the precision of REPOAUDIT.

We present REPOAUDIT as a prototype and power it with Claude 3.5 Sonnet. We test it on three types of critical bugs: null pointer dereference, memory leak, and use-after-free. Our experimental subjects encompass fifteen real-world software systems, with an average size of 251,259 LoC. The results indicate that REPOAUDIT effectively reproduces 21 previously detected bugs and uncovers 17 newly discovered bugs, four of which have already been fixed in the latest commit, achieving the precision of 65.52%. Besides, it is demonstrated that REPOAUDIT exhibits high efficiency and incurs low token costs, averaging 0.44 hours and $2.54 for the analysis of a single software system. Furthermore, we conduct a comparative analysis with two industrial static bug detectors, namely Meta INFER (Meta, 2025) and Amazon CODEGURU (Amazon, 2025). It is found that Amazon CODEGURU reports 18 false positives with no true positives. Meta INFER reports seven true positives two false positives. We also evaluate REPOAUDIT using two more LLMs, namely DeepSeek R1 and GPT-4 Turbo. Particularly, REPOAUDIT powered by DeepSeek R1 detects 44 true bugs and achieves the precision of 75.86%, while it only costs $0.57 for each project on average. Notably, REPOAUDIT facilitates IDE-time code auditing during the development phase, which cannot be supported by Meta INFER and other compilation-dependent bug detectors. To the best of our knowledge, REPOAUDIT is the first purely LLM-driven code auditing technique capable of scaling to real-world software systems. We expect that our efforts will contribute valuable insights to other LLM-driven repository-level coding tasks. We have published bug reports (Guo et al., 2025) and will release the implementation of REPOAUDIT soon.

2. Preliminaries

In this section, we first discuss the essence of repository-level code auditing. Next, we illustrate the limitations of

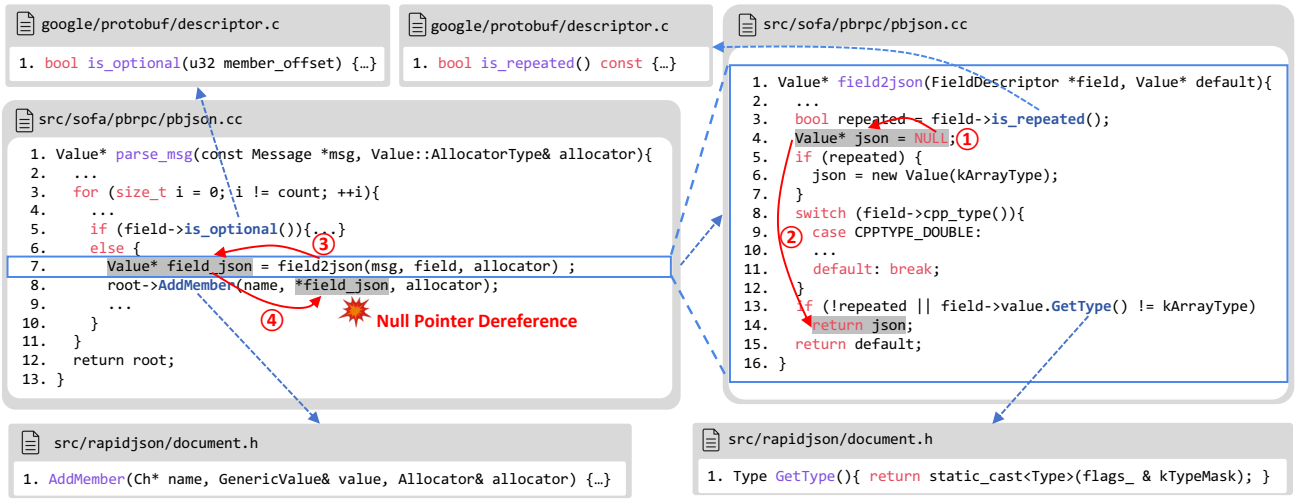


Figure 1. A simplified code snippet from the project `sofa-pbrpc` contains a real NPD bug found by REPOAUDIT. The blue dashed arrows indicate the edges in the call graph. The red solid arrows show the data-flow facts indicating the null value propagation. The call graph of the project contains 1,508 nodes and 6,196 edges, while its data dependence graph contains 160,875 nodes and 360,096 edges.

the LLMs in this task. Finally, we highlight several intrinsic strengths of LLMs in tackling primitive tasks that can be leveraged to build our repository-level auditing tool.

2.1. Path-Sensitive Reasoning on Complex Graphs

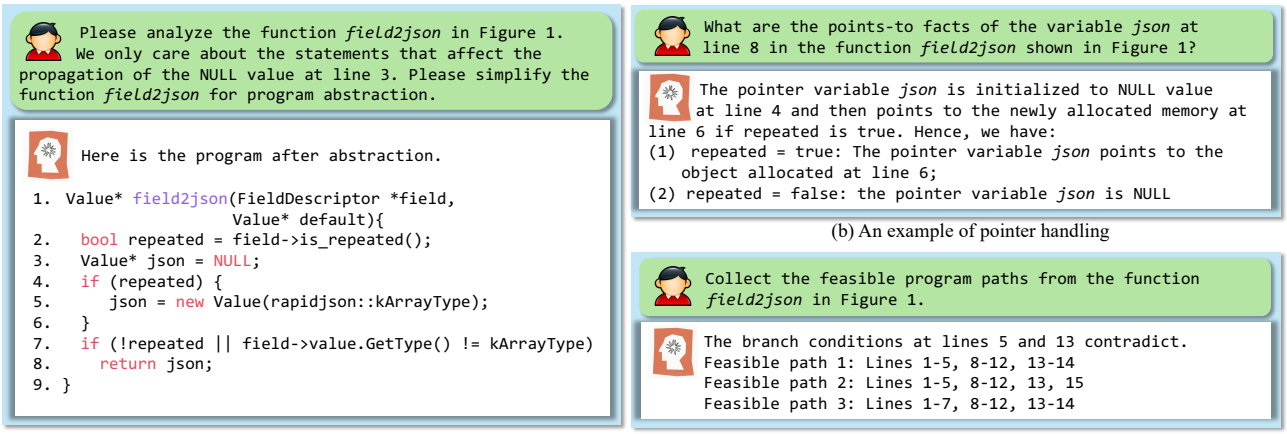
While several bug types, such as API misuse (Li et al., 2021), require only localized reasoning upon abstract syntax trees (ASTs) and are relatively easy to detect with highly effective scanners, many critical bug types demand modeling the entire project as a massive graph and reasoning about properties along and across individual paths in that graph. For example, detecting Null Pointer Dereference (NPD) bugs relies on constructing and analyzing a specialized graph structure called the *data dependence graph* (DDG) (Ferrante et al., 1984), where the nodes represent statements, and the edges indicate *data-flow facts* between different program values at specific statements. Specifically, a data-flow fact exists from the variable u at the statement st_a to the variable v at the statement st_b, denoted as $u@st_a \hookrightarrow v@st_b$, if the value of the variable u at the statement st_a may affect the value of the variable v at the statement st_b following some program path. A *program path* is a sequence of statements that follow the order of execution for some input. It is *feasible* if there exists an input that satisfies all the conditional checks along the path. Consequently, a data-flow fact can occur between two statements that are far apart, such as when a global variable is read and written across different source directories.

Determining whether a data-flow fact may hold in the program requires collecting feasible program paths and analyzing data-flow facts along them. Consider the detection of Null Pointer Dereference (NPD) as an example. Given the DDG, the code auditor ought to find a chain of data-flow facts leading from a null value (as a *source* value) to a dereferenced pointer (as a *sink* value) along a feasible program path. In Figure 1, for instance, the function `field2json` initializes a null value at the line 4 of the function, inducing the data-flow fact $\texttt{NULL}@s_4 \hookrightarrow \texttt{json}@s_4$, which is labeled with ① in Figure 1. When `repeated` is false, the value of `json` at line 4 propagates to the return statement at the line 14. This data-flow fact, denoted by $\texttt{json}@s_4 \hookrightarrow \texttt{json}@s_{14}$, is labeled with ②. Subsequently, in the function `parse_msg`, the return value of `field2json` is assigned to the pointer `field_json` at the line 7 and further dereferenced at the line 8 of the function `parse_msg`, which eventually causes a NPD bug.

Furthermore, we study the 2024 CWE Top 25 Most Dangerous Software Weaknesses, a curated list of the most critical and prevalent vulnerabilities underlying the 31,770 Common Vulnerabilities and Exposures (CVE) records reported in 2024. Our investigation reveals that 19 out of the 25 weakness (76%) categories necessitate *global, path-sensitive reasoning of* ***source-sink reachability*** *upon call graphs, DDGs, and control-flow graphs (CFGs)*, whereas only 6 categories can be effectively identified through localized analysis based on ASTs. Therefore, code auditing for security vulnerability detection necessitates effective path-sensitive reasoning on complex graphs.

2.2. LLMs' Inadequacy

According to (Rozière et al., 2023; OpenAI, 2023), many foundation models are initially pre-trained on relatively short text or code snippets. For long contexts, models like DeepSeek-V3 (Liu et al., 2024), Llama3 series (Dubey



(a) An example of program abstraction

(b) An example of pointer handling

(c) An example of feasible program path exploration

Figure 2. Three examples of showing intrinsic strengths of LLMs

et al., 2024), and QWen2.5-Coder (Hui et al., 2024) typically adopt NTK-aware length interpolation such as YaRN (Peng et al., 2024) to progressively extend context window from the initial 4K/8K to 128K tokens. Although these models perform well in "Needle in a Haystack" (Kamradt, 2023) evaluation, the task is intended for RAG and does not align well with the path-sensitive program understanding ability needed in our application.

To validate our speculation, we conducted a controlled experiment where we prompt Claude 3.5 Sonnet with all the five functions shown in Figure 1 to identify the NPD bug, following a methodology similar to a recent study (Fang et al., 2024a). This is a controlled experiment because, in practice, we cannot guarantee knowledge of the comprehensive set of functions associated with a bug. The model exhibited substantial hallucinations, reporting that almost all the dereferenced pointers have null values. Even if we improve the prompts by offering several few-shot examples and explanations on how a null pointer dereference bug occurs, the model still hallucinates, producing false positives and incorrect explanations. More numerical results can be found in Section 4.3.

2.3. Intrinsic Strengths of LLMs

On the bright side, we observe that LLMs can effectively perform basic analyses when the scope is limited. This capability enables us to surpass traditional program analysis methods, which require compilation and struggle to scale efficiently. Specifically, we identify several primitive abilities critical for auditing: *program abstraction*, *pointer handling*, and *feasible program path exploration*. While traditional auditing tools rely on heavy-weight yet rigorous techniques to achieve these capabilities, skilled human auditors often rely on intuition to handle such challenges within a constrained analysis scope. We observe similar traits in LLMs.

2.3.1. Program Abstraction

Abstraction is essential for scalability in program analysis. Given a property, a set of initial program points, and a defined scope (e.g., a function), abstraction identifies the subset of statements relevant to the property within the scope. These statements form a self-contained, smaller program, significantly reducing the number of paths to analyze. For instance, in the NPD detection (as illustrated in Section 2.1), abstraction targets how null values are propagated in the program and focuses on key statements such as null pointer assignments, value propagations, conditional checks guarding the propagations, and cross-function propagations (e.g., passing the pointer to a callee function via a parameter, returning it to a caller function, or writing it to a global variable). In a preliminary experiment shown in Figure 2(a), for example, we feed the function `field2json` in Figure 1 to Claude 3.5 Sonnet and ask it to abstract the program. Observe that the program returned by the LLM only contains critical statements relating to the null value propagation, while irrelevant ones are removed, such as the switch statement from line 8 to line 12 in the function `field2json` in Figure 1. Notably, human auditors often perform such abstraction implicitly, enabling them to analyze complex code without the excessive path exploration that hampers classic tools like symbolic execution (Cadar et al., 2008) and software model checking (Clarke, 1997).

2.3.2. Pointer Handling

Pointer variables in C and reference variables in Java may point to different memory objects depending on their runtime values. Consequently, reads and writes via pointer dereferences create data-flow facts that dynamically vary based on pointer values. Determining the set of memory objects a pointer variable may point to is known as *points-to analysis* (Smaragdakis et al., 2015), one of the most challenging problems for downstream static analysis, such as

DDG construction. Traditional points-to analysis techniques often rely on conservative and relational methods, which tend to significantly over-approximate the set of possible memory objects, complicating downstream analysis tasks. In contrast, human auditors can often intuitively and accurately determine the points-to facts of a specific pointer variable through their understanding of program semantics. Advanced LLMs exhibit similar capabilities, particularly within the scope of individual functions. As shown in Figure 2(b), for example, we prompt Claude 3.5 Sonnet with the function `field2json` in Figure 1 and query the points-to facts of the return value of the function. The model accurately identifies the two possible points-to facts within the function. It even identifies the path constraints that make the corresponding points-to facts hold. In contrast, the symbolic static analyzer SVF (Sui and Xue, 2016) computes the points-to facts without path conditions by default due to its inherent limitations in semantic analysis.

2.3.3. Feasible Program Path Exploration

A bug report is often considered valid only if it provides a feasible program path (from the root cause to the symptom) as evidence. To determine feasibility, traditional tools rely on modeling conditional checks along a path as symbolic constraints (e.g., in the form of first-order logic formulas) and querying a theorem prover (e.g., SMT solver Z3 (de Moura and Bjørner, 2008)) to check if an input can satisfy the constraints. This process is computationally expensive and prone to failure, as converting program paths to logic formulas requires exploring an explosively large number of program paths and modeling a wide range of program behaviors that lack direct representation in logical term, such as loops, array indexing, aliasing, pointer arithmetic, and unbounded string operations. In contrast, humans, as well as the LLMs, rely on abstraction and intuitive logical reasoning to assess feasibility, which is highly effective within a limited scope. In Figure 2(c), we demonstrate an example of feasible program path exploration upon the function `field2json` using Claude 3.5 Sonnet. Observe that the model can skip the irrelevant statements, such as the switch statement, avoiding exploring a huge number of program paths. It also discovers the contradiction between the branch conditions at lines 5 and 13, which refutes program paths covering both lines 6 and 14, thereby reporting the three feasible program paths in Figure 2(c).

3. RepoAudit

Building on the findings of the previous section, we present RepoAudit, an autonomous LLM-driven agent designed to enable repository-level code auditing by emulating the manual auditing process. The core design rationale is as follows: Since LLMs struggle to reason about path-sensitive properties in large-scale program graphs, RepoAudit employs an agent-centric approach to navigate these graphs externally, prompting the model with one unit (e.g., a function) at a time. This demand-driven navigation adapts based on the model's responses for each function, and a dedicated agent memory ensures that analysis results across functions are seamlessly shared. This approach enables a navigation process that effectively replicates the intended path-sensitive exploration of the original massive graphs. To achieve cost-effective and path-sensitive analysis at the function level, RepoAudit leverages LLMs' intrinsic capabilities by providing explicit prompts for program abstraction, pointer handling, and feasible program path exploration.

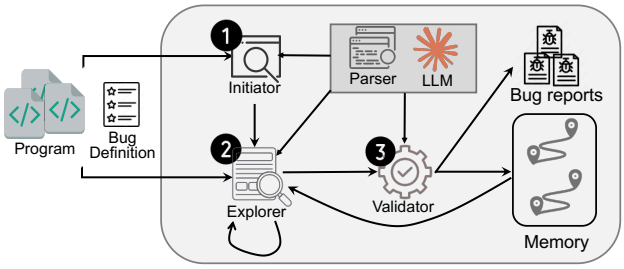


Figure 3. The architecture of RepoAudit

Figure 3 depicts the architecture of our agent. The *initiator* tool (❶) takes a specified bug definition and the target repository as inputs, identifying the source values for analysis, such as the null values for the NPD detection. Each source value triggers a scanning procedure. The *explorer* tool (❷) conducts iterative, demand-driven exploration of the repository by prompting the model to analyze one function at a time. The results of each analysis are stored in the agent memory, guiding further exploration. The analysis prompts are dynamically generated for each function, providing detailed instructions for abstraction tailored to the specific function and adjusted based on the results of prior analyses. While the agent may still hallucinate in the analysis of single functions and produce false positive bug reports, a set of *validator* tools (❸) verify the result of the explorer from multiple perspectives, including the validity of the control flow order and the satisfiability of the path conditions across functions.

3.1. Initiator

As discussed in Section 2.1, detecting the bugs, particularly among the most prevalent CWE bug types, involves traversing paths within specific graphs from well-defined source values as starting points to determine whether any sink values can be reachable. The initiator identifies the starting points of scanning (i.e., the sources). In our implementation, we employ the `tree-sitter` parsing library to create a suite of tailored pattern matchers for the sources of the bug types supported by RepoAudit. These matchers are concise, often consisting of just a few lines of code, and

require a one-time implementation effort. Alternatively, recent advancements offer a promising avenue of synthesizing such matchers using LLMs (Wang et al., 2024a).

3.2. Explorer

For each source value identified by the initiator, the explorer conducts a round of scanning over the repository. During each round, the explorer traverses a subset of functions on demand, beginning with the identified sources. It queries the LLM to analyze one function at a time, storing the results in the agent memory. The explorer performs several actions, each guided by corresponding prompts. These actions include: *analyzing individual functions*, *selecting functions for exploration*, and *generating bug report candidates*. We will introduce the details of the three actions as follows.

3.2.1. Analyzing Individual Functions

As discussed in Section 2.1, scanning for most bug types can be reduced to traversing a limited set of graphs, such as the DDG and the CFG. This enables the use of general analysis prompts for these graph types, eliminating the need for bug-specific prompts. Moreover, instead of explicitly enumerating and analyzing individual paths, we leverage the intrinsic capabilities of LLMs to distinguish paths and perform path-sensitive reasoning. The key of this approach lies in pointer handling and program abstraction demonstrated in Section 2.3.2 and Section 2.3.1, respectively, which implicitly reduces the function to a significantly smaller code snippet with far fewer paths. This can eventually facilitate the efficient exploration of feasible program paths demonstrated in Section 2.3.3. We design the prompt template used for analyzing individual functions in Figure 4. After describing the task at the beginning, we offer the three-step hints to the LLM, enforcing the LLM to unleash its power step-by-step in pointer handling, abstraction, and feasible program path exploration. By offering several few-shot examples, we pose the question at the end and ask the LLM to identify the data-flow facts starting from the initial value(s) of interest along different feasible program paths.

Consider the function `filed2json` and the initial null value at the line 4 as an example. Utilizing the instrinsic strengths of LLMs, RepoAudit collects three feasible program paths shown in Figure 2(c). By simulating the program execution along the first program path, denoted by p_1, the LLM identifies the data-flow facts $\texttt{NULL}@s_4 \hookrightarrow \texttt{json}@s_4$ and $\texttt{json}@s_4 \hookrightarrow \texttt{json}@s_{14}$. For the second and third program paths, denoted by p_2 and p_3, the LLM only identifies the data-flow fact $\texttt{NULL}@s_4 \hookrightarrow \texttt{json}@s_4$.

Agent Memory. After analyzing a function, the explorer obtains a set of data-flow facts for each feasible program path and stores them into the memory of the agent. Specifically, the agent memory is a function $\mathcal{M}$ relating to the function f and a program value $v@s$. Each element in $\mathcal{M}(f, v@s)$ is a pair of a program path and a set of data-flow facts. After analyzing the function `field2json` and the null value at the line 4, for instance, the memory maps $(\texttt{field2json}, \texttt{NULL}@s_4)$ as follows:

$$
\begin{aligned}
&\mathcal{M}\big(\texttt{field2json}, \texttt{NULL}@s_4\big) \\
=&\Big\{\big(p_1, \{\texttt{NULL}@s_4 \hookrightarrow \texttt{json}@s_4, \texttt{json}@s_4 \hookrightarrow \texttt{json}@s_{14}\}\big), \\
&\big(p_2, \{\texttt{NULL}@s_4 \hookrightarrow \texttt{json}@s_4\}\big), \\
&\big(p_3, \{\texttt{NULL}@s_4 \hookrightarrow \texttt{json}@s_4\}\big)\Big\}
\end{aligned}
$$

Role: You are a [LANGUAGE] programmer.
Task: Determine data-flow facts starting from a given value along different program paths in a single function.

Hints: You can collect the feasible program paths into 3 **steps**:
(1) Identify the aliased pointers in the program.
(2) Extract the critical statements relating to the value propagation, like function call statements, return statements, and assignments.
(3) Collect the feasible program paths covering critical statements. For each feasible program path, simulate the program execution and determine the data-flow facts starting from the given value.

Examples: Here are several examples.
Example 1: User: [Program] [Question]
System: <Path information: Lines 4-7, 11> <Explanation>
<Data-flow facts: $ptr@\ell_4 \hookrightarrow qtr@\ell_7, ptr@\ell_4 \hookrightarrow rtr@\ell_{11}$ >
[Other examples]

Question: Given the function [FUNCTION], what are the data-flow facts starting from value [VAL_NAME] at line [VAL_LINE] along different program paths?

Figure 4. The prompt template for analyzing individual functions

3.2.2. Selecting Functions for Exploration

After analyzing a function, if the targeted program value propagates across function boundaries, the explorer queries the underlying call graph to identify the relevant functions for further exploration. This follow-up exploration is guided by the program values escaping the current function boundaries. For example, in Figure 1, the null value at the line 4 of `field2json` is propagated to the return value. Hence, in the next step, the explorer analyzes the caller function of `field2json`, i.e., `parse_msg`, and examines how the return value is propagated.

Note that if the program value does not escape, the analysis does not lead to the explorations of other functions. For example, if we consider the second and third program paths, i.e., p_2 and p_3 in the example shown in Section 3.2.1, the value of `json` at the line 4 of the function `field2json` does not propagate. Hence, the explorer does not enter any other functions. Furthermore, we leverage the existing data-flow facts stored in the memory as caches to avoid redundant analysis. Specifically, before analyzing the propagation of a specific program value in a function, the explorer first checks the agent memory and determines whether this has been done before. In our evaluation, we will quantify how the caching strategy reduces computational costs.

3.2.3. Generating Bug Report Candidates

After analyzing a function, the explorer evaluates whether any new bug candidates can be identified. Consider the NPD detection as an example. The explorer examines whether it identifies any new data-flow facts reaching a sink value, i.e., a dereferenced pointer. If so, a bug report is generated by assembling the complete trail of data-flow facts across functions along with the corresponding inter-procedural program path, which involves linking the data-flow facts stored in the agent memory. For example, the explorer enters the function `parse_msg` and examines how the return value of `field2json` propagates. Based on the two discovered data-flow facts (i.e., the ones labeled as ③ and ④) in Figure 1, the explorer reaches a sink value, i.e., the dereferenced pointer at line 8. Therefore, the explorer identifies a potential bug and report it by concatenation the data-flow facts ①, ②, ③, and ④. For some bug types, such as the ML bugs, the explorer reports a potential bug if it fails to reach any sink point along some feasible path, e.g., the argument of the invocation of `free` function.

3.3. Validator

To improve the quality of bug reports, we introduce two kinds of validation mechanism for the explorer.

Alignment Validation of Data-flow Facts and Control Flow. When dealing with complex code, the LLM may hallucinate and incorrectly infer a data-flow fact $u@s_1 \hookrightarrow v@s_2$ along some program path p that violates the control-flow order. Specifically, this implies that a statement s_2 must appear after another statement s_1 in the program path p, yet the model concludes that a variable defined at s_2 can be used by s_1. To detect such misalignments, a parsing-based analyzer is employed to verify the control-flow order. Only the data-flow facts that conform to the control-flow order will be stored in the agent memory.

Feasibility Validation of Inter-procedural Program Paths. While path feasibility within individual functions is inherently checked by the explorer, contradictions can arise in conditional checks across functions, resulting in infeasible inter-procedural program paths. Recall that each bug report is the concatenation of multiple data-flow facts, such as ①, ②, ③, and ④ in Figure 1. The data-flow facts in different functions may hold under the specific conditions along the corresponding program paths. A bug report candidate is valid only if the path conditions in different functions do not contradict, implying their logical conjunction satisfiable. To check the validity, we prompt the LLM with the inter-procedural path and task it with identifying any contradictions in the path conditions. If a contradiction is found, the bug report is discarded.

Table 1. The statistics of evaluation subjects

Bug Type	ID	Repository Name	Size (LoC)	Stars
NPD	N1	sofa-pbrpc	40,723	2.1K
	N2	ImageMagick/MagickCore	242,555	12.6K
	N3	coturn/src/server	8,976	11.7K
	N4	libfreenect	37,582	3.6K
	N5	openldap	442,955	486
ML	M1	libsass	40,934	4.3K
	M2	memcached	14,654	13.7K
	M3	linux/driver/net	914,025	186K
	M4	linux/sound	1,378,262	186K
	M5	linux/mm	171,721	186K
UAF	U1	Redis	179,723	67.7K
	U2	linux/drivers/peci	2,130	186K
	U3	shadowsocks-libev	71,080	186K
	U4	wabt-tool	3,214	7K
	U5	icu/icu4c/source/i18n	220,359	2.9K

4. Evaluation

We utilize the `tree-sitter` parsing library to provide a set of primitive tools for RepoAudit, e.g., call graph constructor and control flow order validator. We select LLM Claude 3.5 Sonnet to power RepoAudit, the baselines, and the ablations. We evaluate the effectivenss of RepoAudit using two more LLMs, namely GPT-4 Turbo, and DeepSeek R1, and present the main results in the appendix. To reduce the randomness of prompting, we set the temperature to 0.0. Following the common practice in auditing (Heo et al., 2017), we introduce an upper bound K on the calling context and set it to 4, i.e., RepoAudit investigates data-flow facts across a maximum of four functions.

4.1. Bug Types and Dataset

We focus on three kinds bugs, namely NPD, ML, and UAF, which are among the CWE Top 25 more dangerous weaknesses. Our evaluation first aims to reproduce bugs reported in previous works. Specifically, we investigate recent works in the venues of computer security and software engineering and collect the bug reports published by the authors (Huang et al., 2024; Shi et al., 2021; 2018b). Also, we attempt to detect new bugs within the targeted code repositories. As shown by Table 1, we choose five well-maintained projects for each bug type from the bug reports of previous works, which mostly have thousands of stars on GitHub. These projects, with an average size of 251,259 LoC and thousands of functions.

4.2. Performance of RepoAudit

Setup and Metrics. In the detection of NPD bugs, we identify the literal value `NULL` as a source and the dereferenced pointer as a sink. In the detection of ML and UAF bugs, we focus on the memory allocation/free functions used in the reported bugs and identify their return values/arguments via parsing. To reprocedure the bugs reported in the previous

Table 2. The statistics of REPOAUDIT powered by Claude 3.5 Sonnet in the detection of NPD, ML, and UAF bugs. The column "New" separately displays the new bugs identified in the bug-inducing commit and the latest commit. The columns "Intra" and "Inter" show the number of intra-procedural and inter-procedural bugs, respectively.

Bug Type	ID	TP		FP	Feature		# Prompts	# Tokens		Financial ($)	Time (s)
		Old	New		# Intra	# Inter		Input	Output		
NPD	N1	1	(7,7)	2	0	8	145	709,919	55,863	2.97	2026.13
	N2	7	(0,0)	0	4	3	17	97,717	8,518	0.42	283.84
	N3	1	(1,0)	4	1	1	109	599,674	52,936	2.59	1747.90
	N4	1	(0,0)	4	0	1	29	126,852	13,654	0.59	435.09
	N5	1	(3,2)	2	0	4	63	420,710	31,375	1.73	1059.57
ML	M1	1	(2,1)	3	2	1	205	1,132,763	85,279	4.68	2,917.91
	M2	1	(2,2)	2	3	0	146	845,148	71,243	3.60	2282.31
	M3	1	(0,0)	0	1	0	2	10,481	1,019	0.05	34.34
	M4	1	(0,0)	0	1	0	1	5691	619	0.03	17.94
	M5	1	(0,0)	0	1	0	35	181,348	20,779	0.86	599.92
UAF	U1	1	(1,1)	0	1	1	36	179,939	17,547	0.80	582.23
	U2	1	(0,0)	0	1	0	2	8900	869	0.04	31.95
	U3	1	(0,0)	0	1	0	48	317,713	23,067	1.30	791.98
	U4	1	(0,0)	0	1	0	10	48,087	5,883	0.23	185.22
	U5	1	(1,0)	3	1	1	662	4,534,444	303,645	18.15	10,661.98
Average							**100.67**	**614,625.73**	**46,153.07**	**2.54**	**1,577.22**

studies, we check out the repository and analyze the code at the corresponding buggy commit. For newly discovered bugs, we manually check whether they are TPs (TPs) or FPs (FPs), based on which we can obtain the precision of REPOAUDIT. Furthermore, we report the newly discovered bugs to developers if they still exist in the latest version of the repository. To quantify the resource consumption of the code auditing, we measure the input/output token costs, time cost, and financial cost when utilizing REPOAUDIT in analyzing each repository.

Result. As shown in Table 2. The ground-truth report of the project MagickCore contains seven bugs, while the ground truth reports for the other projects each identify one bug. The results demonstrate that REPOAUDITsuccessfully detects all the bugs listed in the ground truth reports. Additionally, REPOAUDIT identifies 17 new bugs, 4 of which have already been fixed in the latest commit, while 13 bugs remain unresolved in the latest version. In total, REPOAUDIT reports 38 TPs and 20 FPs, resulting in a precision of 65.52%. Notably, 20 TPs detected by REPOAUDIT are inter-procedural bugs. By leveraging the data-flow facts uncovered by the LLM, REPOAUDIT can explore the relevant functions and trace the buggy program paths across different functions. Finally, REPOAUDIT demonstrates both efficiency and cost-effectiveness. On average, it takes only 0.44 hours (i.e., 1,577.22 secs) to analyze a program using REPOAUDIT, completing the code auditing within 100.67 rounds of prompting. According to the pricing policy of Claude 3.5 Sonnet, the average cost per project audit is $2.54, with each true bug detection costing $1.03 only.

4.3. Comparison with LLM-driven Bug Detectors

Setup and Metrics. According to our investigation, there are two kinds of LLM-driven code auditing techniques, namely end-to-end few-shot chain-of-thought (CoT) prompting-based approaches and agent-centric approaches. Specifically, the end-to-end few-shot CoT prompting-based approaches can be divided into two categories, including single-function level bug detection and multiple-function level bug detection. First, the single-function level bug detection is widely adopted and evaluated by many recent studies (Chen et al., 2023; Ding et al., 2024). Due to the restricted length of a single function, the context length does not affect the applicability of such technique. To evaluate the single-function level bug detection, we collect all the functions that contain the source values yielding TPs to the LLM along with few-shot examples, asking the LLM to determine whether the function can introduce specific types of bugs. Second, the multiple-function level bug detector attempts to feed the whole program to the LLM so that the calling contexts of buggy functions can be included (Wang et al., 2024b). Unfortunately, the huge size of a real-world software system makes the length of the prompt exceed the context length of the LLM. Hence, we only feed the relevant functions covered by the buggy program paths to the LLM in our evaluation.

Among agent-centric approaches (Wang et al., 2024a; Li et al., 2024a;b), we select LLMDFA (Wang et al., 2024a) as a baseline for evaluation, as it supports compilation-free and customizable analysis. Since LLMDFA can not support the ML detection, we focus our comparison between REPOAUDIT and LLMDFA specifically on the detection of NPD and UAF bugs. Note that LLMDFA necessitates summarizing data-flow facts between sources, parameters, output values, and sinks, arguments, or return values for each function. As a result, the overall computational costs—time, token, and financial costs—can increase significantly. To avoid huge computation costs, we conduct a group of controlled experi-

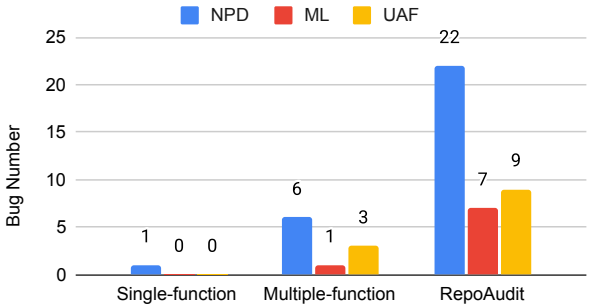


Figure 5. The comparison results with single-function level and multiple-function level bug detectors.

ments under two settings. In the first setting, LLMDFA is applied to functions covered by buggy program paths, generating data-flow summaries for these functions. In the second setting, LLMDFA is tasked with generating data-flow summaries from each source value. We refer to these settings as LLMDFA-PATHSCAN and LLMDFA-SRCSCAN, respectively. Notably, the number of prompting rounds and computational costs of LLMDFA-PATHSCAN and LLMDFA-SRCSCAN are lower than those of LLMDFA, as the latter two only reason a subset of the functions in the repository. By comparing their costs with the one of REPOAUDIT, we can highlight the superior efficiency of our approach relative to existing agent-centric techniques.

Result. Figure 5 shows the comparison results between the single-function level bug detector, multiple-function level bug detector, and REPOAUDIT. The single-function level bug detector can only one intra-procedural NPD bug. The key reasons are twofold. First, it only accesses the last function in the buggy path and lacks the calling context of the function. In NPD detection, for example, the single-function level detector can not determine whether the parameter is null or not, which makes it fail to detect inter-procedural bugs. Second, the model may overlook essential control flows within a function, which results in its inability to accurately identify the data-flow facts of a specific value, thereby achieving low recall in intra-procedural bug detection. For instance, as shown by Listing 1 in Appendix 2, the LLM ignores the error handling branch at line 6, which causes the function to return without releasing the memory object allocated by the function `damon_new_ctx()`, leading to its failure to detect this memory leak bug. With access to the full calling context, the multiple-function level bug detector is able to detect 10 bugs. This demonstrates that providing additional calling context can improve the bug detection capabilities of the LLM. However, the improvement is still limited. Due to hallucinations, the LLM may still wrongly analyze certain intra-procedural and inter-procedural data-flow facts, resulting in a significant number of FNs. In contrast, REPOAUDIT performs the path-sensitive reasoning by employing the program abstraction, which facilitates precisely discovering data-flow facts for code auditing.

Table 3. The ratios of the prompting rounds (**Prompt**) and input token costs (**In_token**) of LLMDFA's ablations and REPOAUDIT

Bug Type	ID	LLMDFA-PATHSCAN		LLMDFA-SRCSCAN	
		Prompt	In_Token	Prompt	In_Token
NPD	N1	115.39	81.23	871.61	522.40
	N2	217.00	120.23	2,839.00	2,087.27
	N3	6.83	3.13	255.57	224.82
	N4	115.76	74.91	778.17	715.23
	N5	199.43	125.53	7,121.86	5,350.62
UAF	U1	41.89	23.08	4,536.75	2,530.74
	U2	24.00	12.29	50.00	26.26
	U3	759.48	693.73	17.63	6.17
	U4	167.50	95.58	46.60	25.42
	U5	4.98	2.09	853.20	447.36
Average		**165.23**	**123.18**	**1737.04**	**1193.63**

Table 3 shows the comparison results between LLMDFA and REPOAUDIT. On average, the number of prompting rounds for LLMDFA-PATHSCAN is 165 times that of REPOAUDIT, and the input token count is 123 times higher. Similarly, for LLMDFA-SRCSCAN, the number of prompting rounds is 1,737 times that of REPOAUDIT, with an input token count 1,193 times higher. It is important to note that the actual cost of LLMDFA would be even greater in practical scenarios. This significant performance gap stems from the fact that LLMDFA conducts pair-wise analyses between specific program values within each function. For example, to detect the NPD bugs in the example program shown in Figure 1, LLMDFA begins with the `NULL` value at line 4 in the function `field2json`, analyzing the dependencies of it with all the arguments, return value and deference pointers (e.g., `field` at line 8) within this function. In contrast, REPOAUDIT requires only two rounds of interaction with the LLM to analyze the functions `field2json` and `parse_msg` for detecting this NPD bug.

4.4. Comparison with Industrial Bug Detectors

Setup and Metrics. We choose two typical static bug detectors as the representatives of industrial tools, namely Meta INFER (Meta, 2025) and Amazon CODEGURU (Amazon, 2025). Specifically, Meta INFER is a static analysis tool from Meta that detects program bugs. Benefiting from its sophisticated memory model (Calcagno et al., 2009), Meta INFER features its outstanding ability in memory bug detection. All three bug types in our evaluation are supported by Meta INFER. Notably, Meta INFER is only applicable to projects that can be successfully compiled. Amazon CODEGURU, as an AWS service, combines machine learning and automated reasoning to identify underlying bugs. Unlike Meta INFER, Amazon CODEGURU can directly analyze the source code without compilation. As Amazon CODEGURU does not support ML detection, we only evaluate it for the NPD and UAF detection on our experimental subjects. After running the two industrial bug detectors, we manually check the bug reports, label the TPs and FPs, and compare the TPs with the ones discovered by REPOAUDIT in Section 4.2.

Table 4. The statistics of Meta INFER and Amazon CODEGURU

Bug Type	ID	Build	INFER		CODEGURU	
			TP	FP	TP	FP
NPD	N1	✗	N/A	N/A	0	0
	N2	✓	1	0	0	0
	N3	✓	N/A	N/A	0	0
	N4	✓	5	0	0	2
	N5	✓	1	2	0	3
ML	M1	✓	0	0	N/A	N/A
	M2	✓	0	0		
	M3	✓	N/A	N/A		
	M4	✓	0	0		
	M5	✓	N/A	N/A		
UAF	U1	✓	N/A	N/A	0	0
	U2	✓	N/A	N/A	0	0
	U3	✗	N/A	N/A	0	0
	U4	✓	0	0	0	0
	U5	✓	0	0	0	13
Total			**7**	**2**	**0**	**18**

Result. Table 4 presents the results of Meta INFER and Amazon CODEGURU. As shown by the column **Build** in Table 4, 13 projects can be successfully compiled However, five of them still cause the crash of Meta INFER, namely the projects with the IDs N3, M3, M5, U1, and U2, even if we have tried multiple versions of Infer, including v1.2.0 (latest), v1.0.0, and v0.9.0. We have reported these issues to Meta INFER, but haven't receibed any response yet. The failures of compiling and analyzing successfully compiled projects demonstrate the restricted applicability and instability of this compilation-dependent bug detector. Eventually, we only successfully analyze eight projects with Meta INFER, obtaining a total of seven TPs and two FPs. Notably, for project `libfreenect` with the project ID N4, the five TPs generated by Infer are based on the assumption that external APIs `realloc` and `malloc` can return null pointers upon allocation failure. However, this assumption does not always hold. In our implementation, REPOAUDIT does not rely on such assumptions and instead begins its analysis with null literal values or other user-defined APIs. Lastly, although Meta INFER is known for its powerful memory model, it still fails to detect any ML or UAF bugs discovered by REPOAUDIT. The unsoundness of industrial static bug detectors make the developers miss many critical bugs in the production line. In comparison, REPOAUDIT not only supports non-compilation analysis with greater applicability but also shows stronger detection capabilities, identifying 38 TPs in total.

As shown by the last two columns in Table 4, Amazon CODEGURU does not detect any TPs upon the targeted 10 projects, while generating 18 FPs. Due to the limitations of its inherent formal reasoning techniques and machine learning models, CODEGURU can only detect certain patterns of bugs and is not robust to semantic-preserving mutations in the code. In contrast, REPOAUDIT leverages LLMs to interpret program semantics, enabling it to discover data-flow facts for effective bug detection.

Table 5. The statistics of REPOAUDIT-NOABS and REPOAUDIT-NOVAL

Bug Type	ID	REPOAUDIT-NOABS			REPOAUDIT-NOVAL		
		TP		FP	TP		FP
		Old	New		Old	New	
NPD	N1	1	(3,3)	7	1	(7,7)	2
	N2	4	(0,0)	7	7	(0,0)	0
	N3	0	(0,0)	6	1	(1,0)	10
	N4	0	(0,0)	0	1	(0,0)	5
	N5	1	(2,2)	2	1	(3,2)	4
ML	M1	1	(1,1)	2	1	(2,1)	3
	M2	1	(1,1)	5	1	(2,2)	5
	M3	1	(0,0)	0	1	(0,0)	0
	M4	0	(0,0)	0	1	(0,0)	1
	M5	0	(0,0)	0	1	(0,0)	2
UAF	U1	1	(0,0)	0	1	(1,1)	0
	U2	1	(0,0)	0	1	(0,0)	0
	U3	1	(0,0)	1	1	(0,0)	0
	U4	1	(0,0)	0	1	(0,0)	0
	U5	0	(1,0)	11	1	(1,0)	9
Total		**13**	**8**	**41**	**21**	**17**	**41**

4.5. Ablation Study

Setup and Metrics. To evaluate the effectiveness of each technical design, we introduce three ablation variants of REPOAUDIT, namely REPOAUDIT-NOABS, REPOAUDIT-NOVAL, and REPOAUDIT-NOCACHE. Specifically, REPOAUDIT-NOABS skips the program abstraction, i.e., removing the second step in the prompt template shown in Figure 4. REPOAUDIT-NOVAL removes the validation of the data-flow facts discovered by the explorer and also skips examining the bug reports. REPOAUDIT-NOCACHE disables the caching strategy upon the agent memory when the explorer analyzes individual functions.

Result. Table 5 presents the results of REPOAUDIT-NOVAL and REPOAUDIT-NOABS. Without the program abstraction, REPOAUDIT-NOABS makes the number of TPs decrease by 44.74%, while the number of the reported FPs increases by 105%, leading to a drop in precision to 33.87%. This decline is attributed to the complex control flows and multiple execution paths created by numerous conditional branches, loop structures, and their nested combinations within functions. When analyzing such cases, the model becomes more susceptible to hallucinations, resulting in missed buggy propagation paths or incorrect identification of non-existing ones. When the validators are disabled, the number of FPs increases to 41, representing an 105% increase. This rise is mainly caused by the presence of various conditional branches and jump statements, such as `if-else` and `switch`, as well as early exits in certain branches (e.g., error handling). Without the validator, the LLM may hallucinate and fail to account for these critical branches, resulting in a large number of spurious data-flow facts that do not meet reachability requirements. Besides, the conflicts between branch conditions discovered by the LLM can also contribute to the high precision.

Table 6. The computational costs of REPOAUDIT-NOCACHE. OOT: the out-of-time, indicating that REPOAUDIT-NOCACHE does not finish the code auditing in 72 hours.

Bug Type	ID	# Prompts		Financial($)		Time(s)	
		Num	Ratio	Num	Ratio	Num	Ratio
NPD	N1	436	3.01	8.65	2.91	6,316.37	3.12
	N2	18	1.06	0.43	1.03	312.01	1.10
	N3	175	1.61	4.12	1.59	3,098.59	1.77
	N4	181	6.24	3.06	5.19	2,342.23	5.38
	N5	137	2.17	4.09	2.37	2,626.21	2.48
ML	M1	459	2.24	10.49	2.24	7,070.68	2.42
	M2	584	4.00	15.83	4.40	10,852.80	4.76
	M3	2	1.00	0.05	0.95	34.34	1.00
	M4	1	1.00	0.03	0.87	20.13	1.12
	M5	200	5.71	4.88	5.67	3,635.66	6.06
UAF	U1	657	18.25	14.34	17.93	11,528.96	19.80
	U2	2	1.00	0.04	1.00	32.78	1.03
	U3	67	1.40	2.10	1.62	1,153.46	1.46
	U4	10	1.00	0.23	0.99	174.43	0.94
	U5	N/A	N/A	N/A	N/A	OOT	N/A
Average		**209.21**	**3.55**	**4.88**	**3.48**	**3,514.19**	**3.75**

The results of REPOAUDIT-NOCACHE are presented in Table 6. For the project `icu`, the analysis time exceeds 72 hours and the number of prompting rounds exceeds 20,000, more than 30 times that of REPOAUDIT. For the remaining projects, on average, the number of prompting rounds increases by 3.55 times, the financial cost by 3.48 times, and the analysis time by 3.75 times compared to REPOAUDIT. Moreover, we observed that the cache hit rate is closely related to the project's size and the density of the DDG. In projects with intricate call graphs and dense DDGs, a single function can appear in multiple propagation paths of faulty values. For example, in the `icu` project, parsing-related functions are hit in the cache 623 times. In such cases, the caching strategy effectively reduces analysis costs and keeps the analysis time within an acceptable range.

4.6. Limitations and Future Works

Although REPOAUDIT demonstrates significant potential in analyzing real-world software systems, it still suffers two major drawbacks. First, the overhead of REPOAUDIT is heavily influenced by the number of sources in the analyzed repository. In the presence of a large number of sources, REPOAUDIT may incur substantial time, token, and financial costs. Second, REPOAUDIT is not sound in detecting inter-procedural data-flow facts, as it only examines flows across a maximum of four functions, which may cause it to miss complex bugs involving longer call chains. In the future, we plan to optimize repository exploration through parallelization, which would reduce the overall time cost of the analysis. Moreover, fine-tuning small models as planners for caller/callee selection is also a promising direction for promoting the repository-level code auditing. The improved efficiency and reduced financial cost would allow for the exploration of repositories with longer call chains.

5. Related Work

LLMs for Code Auditing. A considerable volume of literature has focused on utilizing LLMs for code auditing. In recent years, various benchmarks like BigVul (Fan et al., 2020), PrimeVul (Ding et al., 2024), and DiverseVul (Chen et al., 2023) have been established. Nevertheless, these benchmarks lack calling context for the buggy functions, thereby degrading the validity of such function-level code auditing techniques (Risse and Böhme, 2024). As for repository-level code auditing, existing techniques can generally be categorized into two groups. The first group of techniques harness LLMs to provide specialized pre-knowledge to symbolic code analyzers or examine initial bug reports, while the main body of the code base is scanned by conventional symbolic analyzers (Wang et al., 2024c; Li et al., 2024b;a). One typical example is IRIS (Li et al., 2024b), which employs LLMs to pinpoint sensitive values within programs, aiding traditional analyzers like CODEQL (Avgustinov et al., 2016) in taint-style bug detection. However, due to the dependency on compilation-based symbolic analysis, these techniques lack the capacity for IDE-time analysis without compilation. The second category of approaches utilize LLMs as code interpreters, extracting semantic properties via prompt engineering (Fang et al., 2024a; Wang et al., 2024b;a; Sun et al., 2024). Typically, LLMDFA (Wang et al., 2024a) and LLMSAN (Wang et al., 2024b) incorporate few-shot CoT prompting to discover data-flow paths for bug detection. Our work REPOAUDIT belongs to the latter category, leveraging LLMs for compilation-free semantic analysis along with symbolic methods. Unlike from LLMDFA, REPOAUDIT adopts a demand-driven strategy for codebase exploration, which avoids exhaustive data-flow summary generation for all functions, thereby enhancing analysis scalability. Additionally, REPOAUDIT outperforms LLMSAN in terms of recall by employing individual function analysis instead of repository-level end-to-end prompting, effectively mitigating hallucinations in the overall analysis.

LLM-Agents for Coding Tasks. A substantial body of research concentrate on establishing LLM-agents for a variety of coding tasks, encompassing program repair (Zhang et al., 2024b; Bouzenia et al., 2024), testing (Fang et al., 2024b; Xia et al., 2023), and analysis (Li et al., 2024a; Wang et al., 2024b;a). In contrast to end-to-end solutions, an agent-centric design proves effective in alleviating inherent drawbacks of LLMs, including context limits and hallucinations. Specifically, agent-centric design typically comprises three crucial design aspects. First, many existing works leverage domain-expert knowledge to facilitate program decomposition, enabling agents to address more manageable sub-problems with minimal hallucinations (Wang et al., 2024a; Zhang et al., 2024b; Xia et al., 2024). For example, PYDEX emulates the manual program

debugging and repair process, addressing syntactic and semantic correctness separately (Zhang et al., 2024b). Second, recent studies have endeavored to harness the capabilities of LLMs in planning (Bouzenia et al., 2024; Li et al., 2024a). For instance, agent-based hackers (Google, 2024a;b; Fang et al., 2024b) explore programs by scrutinizing callers and callees based on LLM feedback. RepairAgent, on the other hand, employs LLMs to select appropriate actions for bug comprehension and fixing (Bouzenia et al., 2024). Contrary to the first category, an agent with a planner can autonomously tackle problems with limited human intervention. Third, numerous existing approaches introduce diverse expert tools to agents, effectively mitigating LLM hallucinations. Apart from commonly used tools like Python interpreters and compilers, several agents incorporate manually crafted domain-specific tools, including specialized static analyzers (Li et al., 2024a), and even synthesize tools leveraging general-purpose domain expertise (Wang et al., 2024a; Yang et al., 2024). In this work, RepoAudit emulates the manual code auditing process, breaking down problems into the problem of analyzing data-flow facts in individual functions. Unlike agent-based hackers, RepoAudit maintains data-flow facts in its memory and facilitates the validation for hallucination mitigation. As an agent for repository-level code auditing, its design has the potential to inspire agent-centric design in various other coding tasks.

6. Conclusion

This paper introduces RepoAudit, an autonomous LLM-agent that facilitates precise and efficient repository-level code auditing. By mimicking manual code auditing process, RepoAudit leverages the intrinsic strength of the LLM, such as program abstraction, and conducts the path-sensitive reasoning upon the program in the repository. Our experiments demonstrate the high precision and efficiency of RepoAudit, achieving 65.52% precision and consuming 0.44 hours per repository on average. RepoAudit successfully reproduces 21 previously detected bugs and identifies 17 underlying bugs, 4 of which have already been fixed. Our research showcases the substantial potential of LLM-based code auditing, offering a promising path towards flexible and configurable code security analysis.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, targeting a complicated code-reasoning task, namely repo-level code auditing. We demonstrate the limitations of our work above. We do not expect our work to have a negative broader impact, though leveraging LLMs for repo-level code auditing may come with certain risks, e.g., the leakage of source code in private organizations and potential high token costs. Meanwhile, it is worth more discussions to highlight that our work has the potential to dramatically change the field of software engineering with the power of LLMs. Specifically, LLM-powered code auditing not only enables the analysis of incomplete programs with little customization but addresses other challenges in the code auditing.

First, classical code auditors mainly depend on specific versions of IRs. As a compilation infrastructure evolves, the version of IR code can change, requiring migration of the implementation to support the analysis of new IRs. For example, the Clang compiler has undergone ten major version updates in the past decade, resulting in differences between IRs generated by different compiler versions. The IR differences necessitate tremendous manual efforts to migrate the code auditing implementation for each version update in the long term. However, an LLM-powered code auditing directly operates on the source code and supports different language standards.

Second, classical code auditing lies in the various abstractions and precision settings, especially pointer analysis, a fundamental pre-analysis in the classical code auditing workflow. Specifically, the developers of code auditors have to consider different precision settings and implement the analysis algorithms accordingly. This process demands significant implementation effort. In contrast, LLMs, being aligned with program semantics, serve as interpreters of program semantics and eliminate the need for proposing abstractions and implementing analysis under specific precision settings.

Third, classical code auditing requires reasoning about the semantics of IRs and reimplementing the same algorithm for different languages. In contrast, LLMs serve as general code interpreters and have exceptional performance in understanding short code snippets, no matter which programming languages are used. By following similar prompting strategies, we can easily extend RepoAudit to analyze programs in other languages, including but not limited to C/C++, Python, and JavaScript.

References

Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 8696–8708. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.EMNLP-MAIN.685.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code. *CoRR*, abs/2308.12950, 2023. doi: 10.48550/ARXIV.2308.12950.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Shraddha Barke, Michael B James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):85–111, 2023.

James C. King. Symbolic execution and program testing. *Commun. ACM*, 19(7):385–394, 1976. doi: 10.1145/360248.360252.

Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In Richard Draves and Robbert van Renesse, editors, *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8-10, 2008, San Diego, California, USA, Proceedings*, pages 209–224. USENIX Association, 2008.

Cristiano Calcagno, Dino Distefano, Peter W. O'Hearn, and Hongseok Yang. Compositional shape analysis by means of bi-abduction. In Zhong Shao and Benjamin C. Pierce, editors, *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, pages 289–300. ACM, 2009. doi: 10.1145/1480881.1480917.

Yulei Sui and Jingling Xue. SVF: interprocedural static value-flow analysis in LLVM. In Ayal Zaks and Manuel V. Hermenegildo, editors, *Proceedings of the 25th International Conference on Compiler Construction, CC 2016, Barcelona, Spain, March 12-18, 2016*, pages 265–266. ACM, 2016. doi: 10.1145/2892208.2892235.

Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. Pinpoint: fast and precise sparse value flow analysis for million lines of code. In Jeffrey S. Foster and Dan Grossman, editors, *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018, Philadelphia, PA, USA, June 18-22, 2018*, pages 693–706. ACM, 2018a. doi: 10.1145/3192366.3192418.

Bowen Zhang, Wei Chen, Peisen Yao, Chengpeng Wang, Wensheng Tang, and Charles Zhang. SIRO: empowering version compatibility in intermediate representations via program synthesis. In Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir, editors, *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, pages 882–899. ACM, 2024a. doi: 10.1145/3620666.3651366. URL https://doi.org/10.1145/3620666.3651366.

Anshunkang Zhou, Chengfeng Ye, Heqing Huang, Yuandao Cai, and Charles Zhang. Plankton: Reconciling binary code and debug information. In Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir, editors, *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024-1 May 2024*, pages 912–928. ACM, 2024. doi: 10.1145/3620665.3640382. URL https://doi.org/10.1145/3620665.3640382.

Brittany Johnson, Yoonki Song, Emerson R. Murphy-Hill, and Robert W. Bowdidge. Why don't software developers use static analysis tools to find bugs? In David Notkin, Betty H. C. Cheng, and Klaus Pohl, editors, *35th International Conference on Software Engineering, ICSE '13, San Francisco, CA, USA, May 18-26, 2013*, pages 672–681. IEEE Computer Society, 2013. doi: 10.1109/ICSE.2013.6606613. URL https://doi.org/10.1109/ICSE.2013.6606613.

Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Ryan Tsang, Najmeh Nazari, Han Wang, Houman Homayoun, et al. Large language models for code analysis: Do {LLMs} really do their

job? In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 829–846, 2024a.

Yu Hao, Weiteng Chen, Ziqiao Zhou, and Weidong Cui. E&v: Prompting large language models to perform is by pseudo-code execution and verification. *arXiv preprint arXiv:2312.08477*, 2023.

Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624*, 2024.

Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

Brian Zhang and Zhuo Zhang. Detecting bugs with substantial monetary consequences by llm and rule-based reasoning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Darko Anicic, Sebastian Rudolph, Paul Fodor, and Nenad Stojanovic. Real-time complex event recognition and reasoning–a logic programming approach. *Applied Artificial Intelligence*, 26(1-2):6–57, 2012.

Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.

Xinwei Long, Jiali Zeng, Fandong Meng, Zhiyuan Ma, Kaiyan Zhang, Bowen Zhou, and Jie Zhou. Generative multi-modal knowledge retrieval with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18733–18741, 2024.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186, 2024.

Meta. Infer Static Analyzer. https://fbinfer.com/, 2025. [Online; accessed 29-Jan-2025].

Amazon. Code Review Tool: Amazon CodeGuru Security. https://aws.amazon.com/codeguru/, 2025. [Online; accessed 29-Jan-2025].

Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. Artifact of REPOAUDIT. https://github.com/RepoAuditor/RepoAudit/, 2025. [Online; accessed 29-Jan-2025].

Ziyang Li, Aravind Machiry, Binghong Chen, Mayur Naik, Ke Wang, and Le Song. ARBITRAR: user-guided API misuse detection. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*, pages 1400–1415. IEEE, 2021. doi: 10.1109/SP40001.2021.00090. URL https://doi.org/10.1109/SP40001.2021.00090.

Jeanne Ferrante, Karl J Ottenstein, and Joe D Warren. The program dependence graph and its use in optimization. In *International Symposium on Programming*, pages 125–132. Springer, 1984.

OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. doi: 10.48550/ARXIV.2303.08774.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

Greg Kamradt. Needle in a haystack - pressure testing llms, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack.

Edmund M Clarke. Model checking. In *Foundations of Software Technology and Theoretical Computer Science: 17th Conference Kharagpur, India, December 18–20, 1997 Proceedings 17*, pages 54–56. Springer, 1997.

Yannis Smaragdakis, George Balatsouras, et al. Pointer analysis. *Foundations and Trends® in Programming Languages*, 2(1):1–69, 2015.

Leonardo Mendonça de Moura and Nikolaj Bjørner. Z3: an efficient SMT solver. In C. R. Ramakrishnan and

Jakob Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.

Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. Llmdfa: Analyzing dataflow in code with large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a.

Kihong Heo, Hakjoo Oh, and Kwangkeun Yi. Machine-learning-guided selectively unsound static analysis. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 519–529. IEEE, 2017.

Jianjun Huang, Jianglei Nie, Yuanjun Gong, Wei You, Bin Liang, and Pan Bian. Raisin: Identifying rare sensitive functions for bug detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12, 2024.

Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. Path-sensitive sparse analysis without path conditions. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 930–943, 2021.

Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 693–706, 2018b.

Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. RAID '23, page 654–668, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400707650. doi: 10.1145/3607199.3607242. URL https://doi.org/10.1145/3607199.3607242.

Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, and Xiangyu Zhang. Sanitizing large language models in bug detection with data-flow. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3790–3805, 2024b.

Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. Enhancing static analysis for practical bug detection: An llm-integrated approach. *Proc. ACM Program. Lang.*, 8(OOPSLA1):474–499, 2024a. doi: 10.1145/3649828. URL https://doi.org/10.1145/3649828.

Ziyang Li, Saikat Dutta, and Mayur Naik. Llm-assisted static analysis for detecting security vulnerabilities. *CoRR*, abs/2405.17238, 2024b. doi: 10.48550/ARXIV.2405.17238. URL https://doi.org/10.48550/arXiv.2405.17238.

Jiahao Fan, Yi Li, Shaohua Wang, and Tien N Nguyen. Ac/c++ code vulnerability dataset with code changes and cve summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories*, pages 508–512, 2020.

Niklas Risse and Marcel Böhme. Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection. *arXiv preprint arXiv:2408.12986*, 2024.

Chengpeng Wang, Jipeng Zhang, Rongxin Wu, and Charles Zhang. Dainfer: Inferring API aliasing specifications from library documentation via neurosymbolic optimization. *Proc. ACM Softw. Eng.*, 1(FSE):2469–2492, 2024c. doi: 10.1145/3660816. URL https://doi.org/10.1145/3660816.

Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. QL: object-oriented queries on relational data. In Shriram Krishnamurthi and Benjamin S. Lerner, editors, *30th European Conference on Object-Oriented Programming, ECOOP 2016, July 18-22, 2016, Rome, Italy*, volume 56 of *LIPIcs*, pages 2:1–2:25. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi: 10.4230/LIPIcs.ECOOP.2016.2. URL https://doi.org/10.4230/LIPIcs.ECOOP.2016.2.

Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):1100–1124, 2024b.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134*, 2024.

Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. Teams of llm agents can exploit zero-day vulnerabilities. *arXiv preprint arXiv:2406.01637*, 2024b.

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Universal fuzzing via large language models. *CoRR*, abs/2308.04748, 2023. doi: 10.48550/ARXIV.2308.04748.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

Google. Evaluating Offensive Security Capabilities of Large Language Models. https://googleprojectzero.blogspot.com/2024/06/project-naptime.html, 2024a. [Online; accessed 30-Dec-2024].

Google. Using Large Language Models To Catch Vulnerabilities In Real-World Code. https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html, 2024b. [Online; accessed 30-Dec-2024].

Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi R. Fung, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Yiquan Wang, Heng Ji, and Chengxiang Zhai. If LLM is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents. *CoRR*, abs/2401.00812, 2024. doi: 10.48550/ARXIV.2401.00812. URL https://doi.org/10.48550/arXiv.2401.00812.

A. Additional Experimental Results

Table 7 and Table 8 show the results of REPOAUDIT powered by DeepSeek R1 and GPT-4 Turbo separately.

With DeepSeek R1, REPOAUDIT successfully detects all the bugs listed in the ground truth reports and identifies 44 true positives, achieving an accuracy of 75.86%. This is because DeepSeek R1 is a reasoning model capable of automatically generating reasoning processes based on the problem, allowing it to better model a program's control-flow and data-flow. According to DeepSeek R1's pricing policy, the average cost per project analysis is only $0.57, the lowest among the three LLMs. However, due to the need to first generate a reasoning chain, DeepSeek R1 requires a longer analysis time.

GPT-4 Turbo performs the worst among the three models, detecting only 14 true positives while generating 25 false positives, resulting in an accuracy of 35.90%. Additionally, GPT-4 Turbo is the most expensive, priced at $30 / 1M tokens, leading to an average analysis cost of $3.47 per project.

Overall, DeepSeek R1 is the best-performing model among the three, with the highest detection capability and the lowest financial cost, though it has a relatively longer analysis time. Claude 3.5 Sonnet performs slightly worse than DeepSeek R1 but completes the analysis in one-third of the time. Meanwhile, GPT-4 Turbo exhibits the worst performance.

Table 7. The statistics of REPOAUDIT powered by DeepSeek R1 in the detection of NPD, ML, and UAF bugs. The column "New" separately displays the new bugs identified in the bug-inducing commit and the latest commit. The columns "Intra" and "Inter" show the number of intra-procedural and inter-procedural bugs, respectively.

Bug Type	ID	TP		FP	Feature		# Prompts	# Tokens		Financial ($)	Time (s)
		Old	New		# Intra	# Inter		Input	Output		
NPD	N1	1	(7,7)	2	0	8	136	675,711	46,839	1.20	12,378.10
	N2	7	(2,0)	0	5	4	14	74,918	6,057	0.14	1,588.65
	N3	1	(1,0)	2	1	1	97	524,915	38,644	0.95	9,044.85
	N4	1	(0,0)	0	0	1	7	29,898	2,288	0.05	247.99
	N5	1	(2,2)	1	0	3	43	270,880	15,689	0.47	2,936.52
ML	M1	1	(4,4)	5	2	3	165	910,153	55,623	1.58	14,633.28
	M2	1	(5,5)	2	3	3	93	562,470	35,498	0.98	5,583.13
	M3	1	(0,0)	0	1	0	2	10,481	832	0.02	74.01
	M4	1	(0,0)	0	1	0	1	5691	214	0.01	93.23
	M5	1	(0,0)	0	1	0	18	92,573	8,289	0.18	1,572.31
UAF	U1	1	(0,0)	0	1	0	50	255,048	19,494	0.46	2,444.72
	U2	1	(0,0)	0	1	0	2	8900	704	0.02	88.76
	U3	1	(0,0)	0	1	0	29	227,336	10,269	0.37	2,768.59
	U4	1	(0,0)	0	1	0	11	53,647	3,617	0.09	1,997.20
	U5	1	(1,0)	2	2	0	226	1,191,391	78,226	2.10	13,124.44
Average							59.60	326,267.47	21,485.53	0.57	4,571.72

Table 8. The statistics of REPOAUDIT powered by GPT-4 Turbo in the detection of NPD, ML, and UAF bugs. The column "New" separately displays the new bugs identified in the bug-inducing commit and the latest commit. The columns "Intra" and "Inter" show the number of intra-procedural and inter-procedural bugs, respectively.

Bug Type	ID	TP		FP	Feature		# Prompts	# Tokens		Financial ($)	Time (s)
		Old	New		# Intra	# Inter		Input	Output		
NPD	N1	1	(0,0)	2	0	1	52	247,272	27,360	3.29	1,459.28
	N2	4	(0,0)	0	2	2	24	155,023	14,286	1.98	522.47
	N3	0	(0,0)	1	0	0	73	394,790	39,803	5.14	1,598.64
	N4	1	(0,0)	0	0	1	17	80,877	8,169	1.05	299.13
	N5	0	(0,0)	2	0	0	12	85,252	6,558	1.05	268.47
ML	M1	1	(1,0)	2	2	0	265	1336994	115376	16.83	5,740.51
	M2	0	(1,1)	6	0	1	92	566135	48320	7.11	2,033.85
	M3	1	(0,0)	0	1	0	2	10481	1134	0.14	44.26
	M4	0	(0,0)	0	0	0	1	5691	811	0.08	30.78
	M5	0	(0,0)	0	0	0	13	66601	7672	0.90	330.96
UAF	U1	1	(0,0)	3	1	0	18	93179	9641	1.22	425.85
	U2	0	(0,0)	0	0	0	2	8900	958	0.12	74.49
	U3	1	(0,0)	3	1	0	31	251232	18125	3.06	2,238.58
	U4	1	(0,0)	0	1	0	10	48087	4044	0.60	188.15
	U5	0	(1,0)	6	0	0	146	749607	68023	9.54	3,369.90
Average							50.53	273,341.40	24,685.33	3.47	1,241.69

B. Case Study

B.1. Hallucination of LLM-driven Bug Detectors

In Listing 1, the function `damon_new_ctx` allocates a memory object at line 2. When function `damon_select_ops` returns false, the function `damon_reclaim_init` exits without releasing the memory allocated at line 2, causing a memory leak bug. However, the LLM-driven bug detectors ignore the error handling branch at line 6, leading to its failure to detect this bug.

Listing 1. An example of ignoring control flow facts within a single function.

```
 1. static int __init damon_reclaim_init(void){
 2.   ctx = damon_new_ctx();
 3.   if (!ctx)
 4.      return -ENOMEM;
 5.   if (damon_select_ops(ctx, DAMON_OPS_PADDR))
 6.      return -EINVAL;
 7.   ctx->callback.after_wmarks_check = damon_reclaim_after_wmarks_check;
 8.   ctx->callback.after_aggregation = damon_reclaim_after_aggregation;
 9.   target = damon_new_target();
10.  if (!target) {
11.     damon_destroy_ctx(ctx);
12.     return -ENOMEM;
13.  }
14.  damon_add_target(ctx, target);
15.  schedule_delayed_work(&damon_reclaim_timer, 0);
16.  damon_reclaim_initialized = true;
17.  return 0;
18. }
```

B.2. True Positives Detected by Meta INFER

We present true positive examples detected by Meta INFER that rely on specific assumptions. Because REPOAUDIT is not specified with the same assmuptions, it can not identify the source values with the initiator, making REPOAUDIT miss the bugs detected by Meta INFER.

Case I: Assume *malloc* Returns Null Value

In Listing 2, Meta INFER assumes that the function `malloc` returns a null value at line 3. Based on this assumption, it reports `kinect->dev` as a null pointer derefernece bug.

Listing 2. An example of a true positive caused by the potential null return value of **malloc**.

```
 1. static sync_kinect_t *alloc_kinect(int index)
 2. {
 3.  sync_kinect_t *kinect = (sync_kinect_t*)malloc(sizeof(sync_kinect_t));
 4.  if (freenect_open_device(ctx, &kinect->dev, index) < 0) {
 5.   free(kinect);
 6.   return NULL;
 7.  }
 8.  int i;
 9.  for (i = 0; i < 3; ++i) {
10.   kinect->video.bufs[i] = NULL;
11.   kinect->depth.bufs[i] = NULL;
12.  }
13.  kinect->video.fmt = -1;
14.  kinect->video.res = -1;
15.  kinect->depth.fmt = -1;
16.  kinect->depth.res = -1;
17.  freenect_set_video_callback(kinect->dev, video_producer_cb);
18.  freenect_set_depth_callback(kinect->dev, depth_producer_cb);
19.  pthread_mutex_init(&kinect->video.lock, NULL);
20.  pthread_mutex_init(&kinect->depth.lock, NULL);
21.  pthread_cond_init(&kinect->video.cb_cond, NULL);
22.  pthread_cond_init(&kinect->depth.cb_cond, NULL);
23.  return kinect;
24. }
```

Case II: Assume *realloc* Returns Null Value.

In Listing 3, Meta INFER assumes the function `realloc` returns a null value at line 8 and identifies `out[pos++]` as the sink of a null pointer derefernece bug.

Listing 3. An example of a true positive caused by the potential null return value of **realloc**.

```
 1. static char *one_line(FILE *fp)
 2. {
 3.  int c;
 4.  int pos = 0;
 5.  char *out = NULL;
 6.  for (c = fgetc(fp); !(c == '\n' || c == EOF); c = fgetc(fp))
 7.  {
 8.   out = realloc(out, pos + 1);
 9.  out[pos++] = c;
10.  }
11.  if (out) {
12.   out = realloc(out, pos + 1);
13.   out[pos] = '\0';
14.  }
15.  return out;
16. }
```